

Genre-Based User Clustering for Movie Recommendation

Group 8

September 29, 2026

Abstract

The rapid growth of online movie platforms has made it difficult for users to find movies that match their personal interests. In this project, we apply unsupervised machine learning to group users according to their genre preferences. Instead of representing each user by thousands of movie ratings, we transform the original rating data into a compact User×Genre matrix where each cell stores a mean-centered genre preference score to remove individual rating bias. K-Means clustering is applied to identify groups of users with similar tastes. The selected number of clusters, $K = 5$, is chosen using multiple criteria rather than a single metric: SSE/Elbow behavior, Silhouette Score, cluster-size behavior, and stability across random seeds. After clustering, a cluster-based recommendation module collects the ratings of users in the same cluster, aggregates unseen movies, filters them by a minimum number of ratings, and ranks them with a weighted score that combines average rating and rating support. New users and weak-profile users are handled by fallback logic. The main machine learning task of this project is user clustering, while the recommendation engine is an application built on top of the clustering result.

Contents

1	Introduction	3
1.1	About Dataset	3
1.2	Motivation	3
1.3	Contribution	3
2	Related Work	4
2.1	Existing Work	4
2.2	Our Work	4
3	Project Process	4
3.1	Data Preprocessing	4
3.2	Centered Rating	5
3.3	User-Genre Matrix	5
3.4	Feature Scaling	6
3.5	K-Means Clustering	7
3.6	Cluster-Based Movie Recommendation	8
4	Dataset Analysis	9
4.1	Rating Distribution	9
4.2	User Rating Behavior	10
4.3	Genre Distribution	10

5	Experiments	11
5.1	Experimental Protocol	11
5.2	Selection of K	12
5.3	Cluster Interpretation	14
5.4	Recommendation Demonstration	14
5.5	Clustering Evaluation	15
5.6	Limitations	16
5.7	Future Work	16

1 Introduction

Recommendation systems are widely used to help users discover relevant items from large catalogs. A common approach is collaborative filtering, which recommends items based on user-item interactions such as ratings. However, movie rating datasets are usually sparse: in the MovieLens small dataset, 610 users and 9,742 movies produce only 100,836 observed ratings, making direct user comparison difficult. Raw ratings also contain user bias, where some users consistently rate higher or lower than others.

To address these issues, this project summarizes user behavior into a 19-dimensional User-Genre matrix with mean-centered ratings to remove rating-scale bias. K-Means clustering groups users with similar genre preferences. For recommendation, the unseen movies rated by users in the same cluster are aggregated, filtered, and ranked using a weighted score.

1.1 About Dataset

The MovieLens small dataset [1] contains 100,836 ratings from 610 users across 9,742 movies. Each movie is labeled with one or more genres, such as Action, Comedy, Drama, Romance, Sci-Fi, Thriller, and others. The main files used in this project are:

- `movies.csv`: contains movie identifiers, titles, and genres.
- `ratings.csv`: contains user identifiers, movie identifiers, rating values, and timestamps.
- `tags.csv`: contains user-generated tags.
- `links.csv`: contains external movie identifiers.

This project uses only `movies.csv` and `ratings.csv`, since the goal is to learn user genre preferences from rating behavior. Tags are not part of the final feature matrix.

1.2 Motivation

Traditional user-movie rating data is usually sparse because each user rates only a small number of movies compared to the full movie catalog. In this dataset, the full user-movie matrix would contain $610 \times 9,742$ possible entries, but only 100,836 ratings are available.

To reduce sparsity and make the representation more meaningful, we transform movie-level ratings into genre-level preferences. This is a domain-based feature extraction step. Instead of using thousands of movie features, each user is represented using 19 genre features. This lower-dimensional representation is more suitable for clustering and easier to explain.

1.3 Contribution

The main contributions of this project are:

- We reduce the sparse user-movie rating matrix into a compact User-Genre matrix, which makes clustering more feasible and interpretable.
- We apply mean-centering so that user profiles reflect genre preference relative to each user's own rating habit, rather than only raw rating scale.
- We use K-Means clustering as the main modeling approach to identify groups of users with similar genre-preference patterns.
- We select the number of clusters using multiple criteria: SSE, Silhouette Score, cluster-size analysis, and multi-seed stability analysis, and interpret the resulting user clusters.

- We build a cluster-based recommendation module as a downstream application: unseen movies from users in the same cluster are aggregated, filtered, and ranked using a weighted score, with fallback logic for new and weak-profile users.

2 Related Work

2.1 Existing Work

Lee, Kim, Lee, and Hwang (2012) [2] reviewed and classified recommendation system research into major methodological categories, including collaborative filtering, content-based recommendation, hybrid recommendation, and knowledge-based approaches. Their survey highlights common challenges in recommender systems, such as data sparsity, scalability, evaluation, and the difficulty of building accurate recommendations when user-item interactions are limited. These challenges directly motivate our approach, as the MovieLens rating matrix is sparse and high-dimensional.

2.2 Our Work

Rather than directly applying collaborative filtering in the original user-movie space, we use genre information to construct a lower-dimensional representation that is easier to cluster and interpret.

3 Project Process

Our pipeline consists of the following steps (Figure 1):



Figure 1: Overall methodology pipeline

3.1 Data Preprocessing

The system uses two main CSV files from the MovieLens small dataset: `movies.csv` and `ratings.csv`. The movie file contains 9,742 movies with three main columns: `movieId`, `title`, and `genres`. The rating file contains 100,836 ratings from 610 users on a scale from 0.5 to 5.0.

Table 1: MovieLens small dataset statistics	
Metric	Value
Total movies	9,742
Total users	610
Total ratings	100,836
Rating scale	0.5–5.0
Number of genres	19
Average ratings per user	≈ 165
Movies rated per user	min 20, max 2,698

The `genres` attribute in the MovieLens dataset is stored as a pipe-separated string, such as:

Adventure|Animation|Children|Comedy|Fantasy

To convert this categorical information into numerical features, the genre string is transformed into binary indicator variables using one-hot encoding. Each genre becomes a separate column, where a value of 1 indicates that the movie belongs to that genre and 0 otherwise.

Table 2: Example of one-hot genre encoding

Title	Action	Adventure	Animation	Comedy	Fantasy
Toy Story (1995)	0	1	1	1	1
Jumanji (1995)	0	1	0	0	1
Heat (1995)	1	0	0	0	0

After one-hot encoding, the genre indicators are merged with the rating table using `movieId`. The resulting data contains one row for each user-movie rating event, together with the rating value and the corresponding movie genre indicators.

3.2 Centered Rating

Using raw ratings directly can be misleading because different users may have different rating habits. Some users tend to give high scores to most movies, while others are stricter and usually give lower scores. If raw ratings are used directly, clustering may group users by rating scale instead of real movie taste.

To reduce this user bias, we compute a centered rating for each user and genre. For user u and genre g , the centered genre preference is defined as:

$$\tilde{r}_{u,g} = \bar{r}_{u,g} - \bar{r}_u \quad (1)$$

where $\bar{r}_{u,g}$ is the mean raw rating that user u gives to movies belonging to genre g , and $\bar{r}_u$ is the overall mean rating of user u across all rated movies.

Formally, let $M_{u,g}$ be the set of movies rated by user u that belong to genre g , and let M_u be the set of all movies rated by user u . Then:

$$\bar{r}_{u,g} = \frac{1}{|M_{u,g}|} \sum_{m \in M_{u,g}} r_{u,m}, \quad \bar{r}_u = \frac{1}{|M_u|} \sum_{m \in M_u} r_{u,m} \quad (2)$$

If user u has not rated any movie in genre g , the centered preference is set to 0, which represents a neutral value.

A positive value $\tilde{r}_{u,g} > 0$ means that user u rates genre g above their personal average, indicating a stronger preference. A negative value means that the user rates that genre below their personal average.

Table 3: Centered rating examples for selected users and genres

User	$\bar{r}_u$	Action	Animation	Comedy	Drama	Romance	Thriller
1	4.37	-0.044	0.323	-0.089	0.163	-0.059	-0.221
2	3.95	0.006	0.000	0.052	-0.066	0.552	-0.248
3	2.44	1.136	-1.936	-1.436	-1.686	-1.936	1.707
4	3.56	-0.236	0.444	-0.046	-0.072	-0.176	-0.003
5	3.64	-0.525	0.697	-0.170	0.164	-0.545	-0.081

3.3 User-Genre Matrix

Using the centered genre preferences, we construct a User-Genre matrix:

$$R \in \mathbb{R}^{N \times G} \quad (3)$$

where $N = 610$ is the number of users and $G = 19$ is the number of genres. Each entry $R_{u,g}$ represents the centered preference of user u for genre g . Each row of the matrix is a 19-dimensional preference vector:

$$R_u = [R_{u,1}, R_{u,2}, \dots, R_{u,19}] \quad (4)$$

This vector represents the movie taste of a user across all genres. Compared with the original User-Movie matrix, which has 9,742 movie dimensions, the User-Genre matrix is much smaller and easier to interpret.

Table 4: Properties of the User-Genre matrix

Property	Value
Rows	610 users
Columns	19 genres
Value range	$[-3.54, 2.56]$
Neutral fill value	0.0
Active-user threshold	10 ratings
Active users retained	610

Users with fewer than 10 total ratings are considered weak-profile users because their genre vectors may be too sparse. However, in this dataset, all 610 users have at least 20 ratings, so all users are retained for clustering.

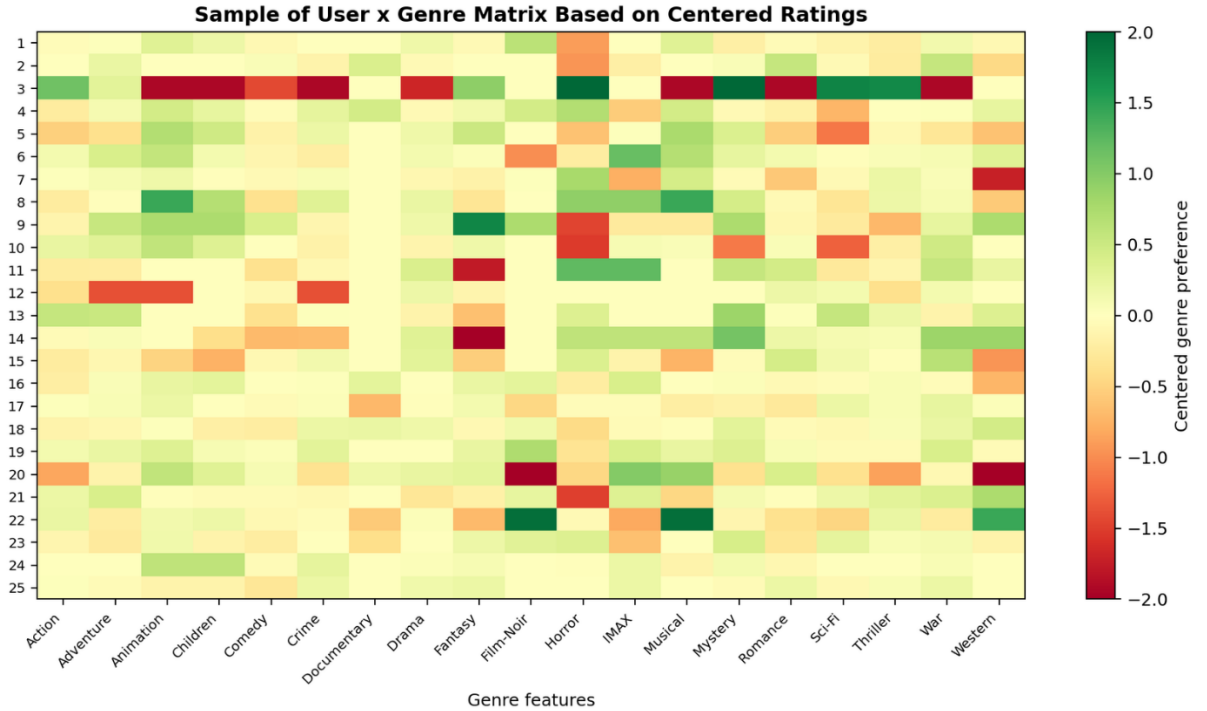


Figure 2: Sample of the User-Genre matrix based on centered ratings

3.4 Feature Scaling

Before applying K-Means, the User-Genre matrix is standardized using z-score normalization. This step is necessary because K-Means uses Euclidean distance to assign samples to clusters.

If one genre feature has a larger variance than others, it may dominate the distance calculation. For each genre feature g , the standardized value is computed as:

$$z_{u,g} = \frac{R_{u,g} - \mu_g}{\sigma_g} \quad (5)$$

where μ_g and σ_g are the mean and standard deviation of genre g computed over all users. After scaling, each genre column has approximately zero mean and unit variance. The scaled matrix is denoted as:

$$Z \in \mathbb{R}^{610 \times 19} \quad (6)$$

This scaled matrix is used as the input to the K-Means clustering algorithm.

Table 5: Effect of StandardScaler on selected genre columns

Genre	Before Scaling		After Scaling	
	μ_g	σ_g	μ_g	σ_g
Action	-0.102	0.345	0.000	1.000
Animation	-0.004	0.600	0.000	1.000
Documentary	0.062	0.431	0.000	1.000
Film-Noir	0.089	0.480	0.000	1.000

3.5 K-Means Clustering

K-Means partitions the 610 users into K clusters by minimizing the Within-Cluster Sum of Squares:

$$\text{WCSS}(K) = \sum_{k=1}^K \sum_{u: y_u=k} \|z_u - c_k\|^2 \quad (7)$$

where z_u is the scaled genre preference vector of user u , c_k is the centroid of cluster k , and y_u is the cluster assignment of user u . The centroid of each cluster is computed as the mean of all users assigned to that cluster:

$$c_k = \frac{1}{|C_k|} \sum_{u: y_u=k} z_u \quad (8)$$

The K-Means algorithm follows two main steps:

1. **Assignment step:** each user is assigned to the nearest centroid:

$$y_u = \arg \min_k \|z_u - c_k\|^2 \quad (9)$$

2. **Update step:** each centroid is updated as the mean vector of all users assigned to that cluster:

$$c_k = \frac{1}{|C_k|} \sum_{u: y_u=k} z_u \quad (10)$$

These two steps are repeated until the cluster assignment becomes stable or the stopping condition is reached. In this project, K-Means is implemented using `sklearn.cluster.KMeans` with `n_init = 10` and `random_state = 42`. To select the number of clusters, we test K from 2 to 14 and evaluate each value using SSE (Elbow behavior), Silhouette Score, cluster-size behavior, and stability across random seeds (Section 5.2). No single metric is treated as decisive. $K = 5$ was selected as the best practical trade-off among clustering quality, stability, cluster size, interpretability, and recommendation usability.

Table 6: Cluster sizes and dominant genres for $K = 5$ (selected final run)

Cluster	Users	Dominant genres
0	107	Crime, Film-Noir, Drama, Mystery, War
1	279	War, Animation, IMAX, Film-Noir, Musical
2	63	Fantasy, Animation, Children, Adventure, IMAX
3	25	War, Drama, Mystery, Film-Noir, Documentary
4	136	Mystery, Thriller, IMAX, Crime, Sci-Fi
Total	610	

Each centroid represents the average genre-preference profile of a group of users. The five clusters therefore represent different movie-taste patterns. Cluster 0 shows relatively stronger preferences for Crime, Film-Noir, Drama, Mystery, and War. Cluster 1 is the largest and represents a broader user group, with relatively strong preferences for genres such as War, Animation, IMAX, Film-Noir, and Musical. Cluster 2 is characterized by Fantasy, Animation, Children, Adventure, and IMAX, indicating a stronger preference for family-oriented and fantasy-related movies. Cluster 3 shows stronger preferences for War, Drama, Mystery, Film-Noir, and Documentary. Finally, Cluster 4 is characterized by Mystery, Thriller, IMAX, Crime, and Sci-Fi, representing users with stronger preferences for suspenseful and science-fiction content.

3.6 Cluster-Based Movie Recommendation

After clustering, each user is assigned to a group of users with similar genre-preference patterns. The recommendation module is built on top of these clusters. For an existing active user u assigned to cluster C_k , candidate movies are obtained from ratings provided by other users in the same cluster.

Candidate generation and filtering. For each candidate movie m , we compute the number of ratings contributed by other users in the target cluster:

$$\text{rating_count}(m) = |\{v \in C_k \setminus \{u\} : v \text{ has rated } m\}|. \quad (11)$$

The corresponding average rating is

$$\text{avg_rating}(m) = \frac{1}{\text{rating_count}(m)} \sum_{\substack{v \in C_k \setminus \{u\} \\ v \text{ rated } m}} r_{v,m}. \quad (12)$$

Let $W(u)$ denote the set of movies already watched by user u . Movies already watched by the target user are removed. To avoid recommendations based on very weak evidence, a candidate must also receive at least τ ratings from users in the cluster, where the current implementation uses $\tau = 3$. The final candidate set is therefore

$$M_{\text{cand}} = \{m : \text{rating_count}(m) \geq \tau\} \setminus W(u). \quad (13)$$

Weighted scoring and Top- N ranking. Each remaining candidate is ranked using

$$\text{score}(m) = \text{avg_rating}(m) \log(1 + \text{rating_count}(m)). \quad (14)$$

Using only the average rating may overvalue a movie supported by very few ratings. The logarithmic rating-count term adds confidence from additional ratings while reducing the dominance of highly popular movies. The candidate movies are sorted by this score in descending order, and the top- N movies are returned.

User cases. The recommendation strategy depends on the amount of information available for the user, as summarized in Table 7.

Table 7: Recommendation strategy for different user cases

User case	Recommendation strategy
Existing active user	Use the assigned K-Means cluster, exclude watched movies, and rank candidate movies using the weighted score.
New user with favorite genres	Construct an approximate preference vector from the selected genres, standardize it, assign it to the nearest K-Means centroid, and recommend movies from that cluster.
New user without favorite genres	Use the popularity-based fallback because no preference information is available.
Weak-profile user	Use the popularity-based fallback because the available rating history is insufficient for a reliable genre-preference profile.

4 Dataset Analysis

4.1 Rating Distribution

Figure 3 shows the distribution of rating values in the dataset. Most ratings are concentrated around 3.0, 3.5, and 4.0, indicating that users tend to give medium to high ratings more frequently than very low ratings.

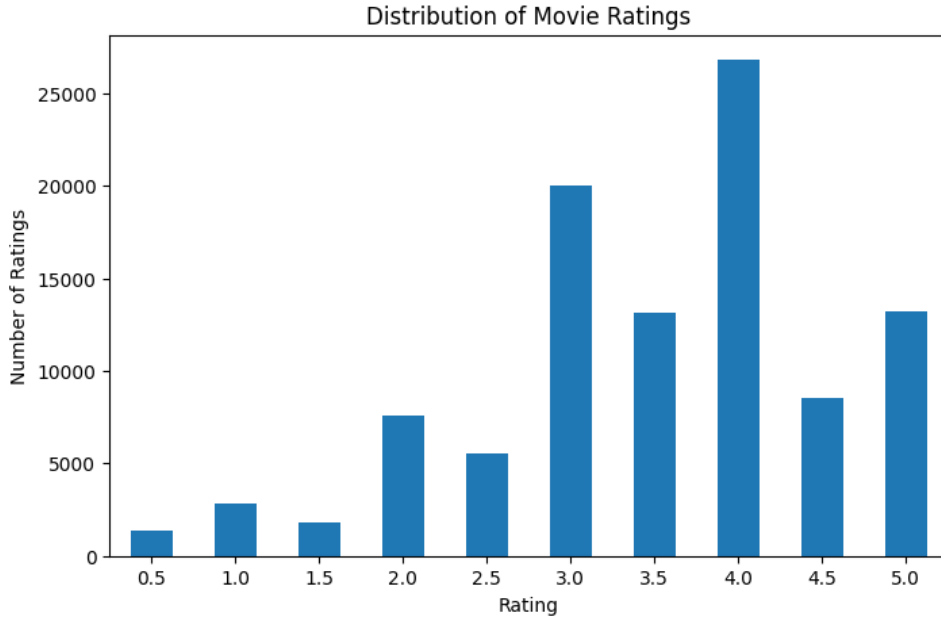


Figure 3: Rating Distribution

4.2 User Rating Behavior

Figure 4 shows that the number of ratings per user is highly uneven. Most users rate relatively few movies, while a small number of users contribute many ratings. This motivates a more comparable user representation: genre-level aggregation reduces sparsity, while mean-centering reduces differences in individual rating scales before clustering.

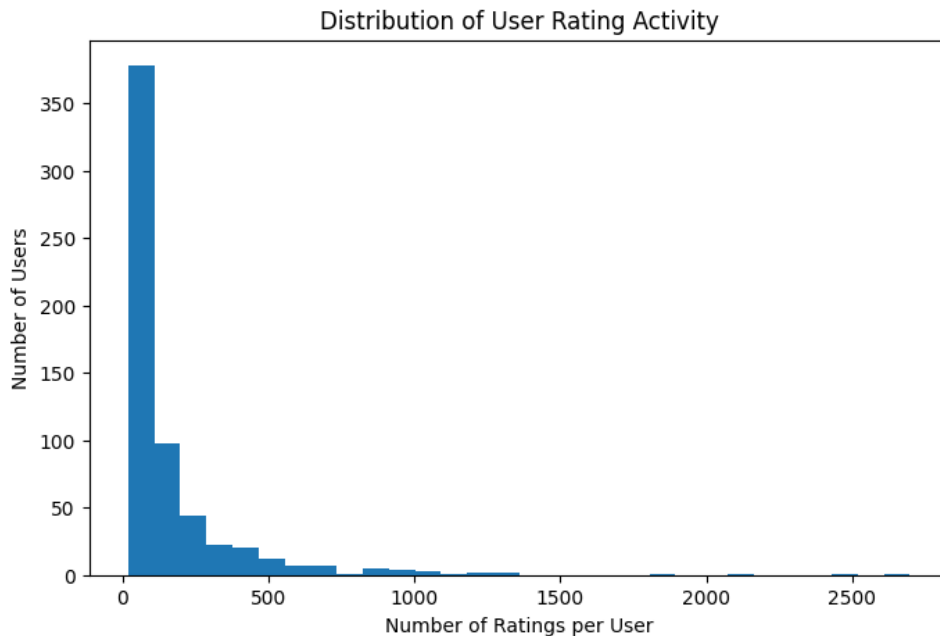


Figure 4: Distribution of the Number of Ratings per User

4.3 Genre Distribution

Figure 5 shows the heavy imbalance across genres. Drama and Comedy dominate with over 4,000 and 3,700 movies, respectively, while genres like Film-Noir, IMAX, and Western have fewer than 200 movies. This imbalance may influence the learned user profiles because dominant genres are represented by substantially more movies and ratings than niche genres.

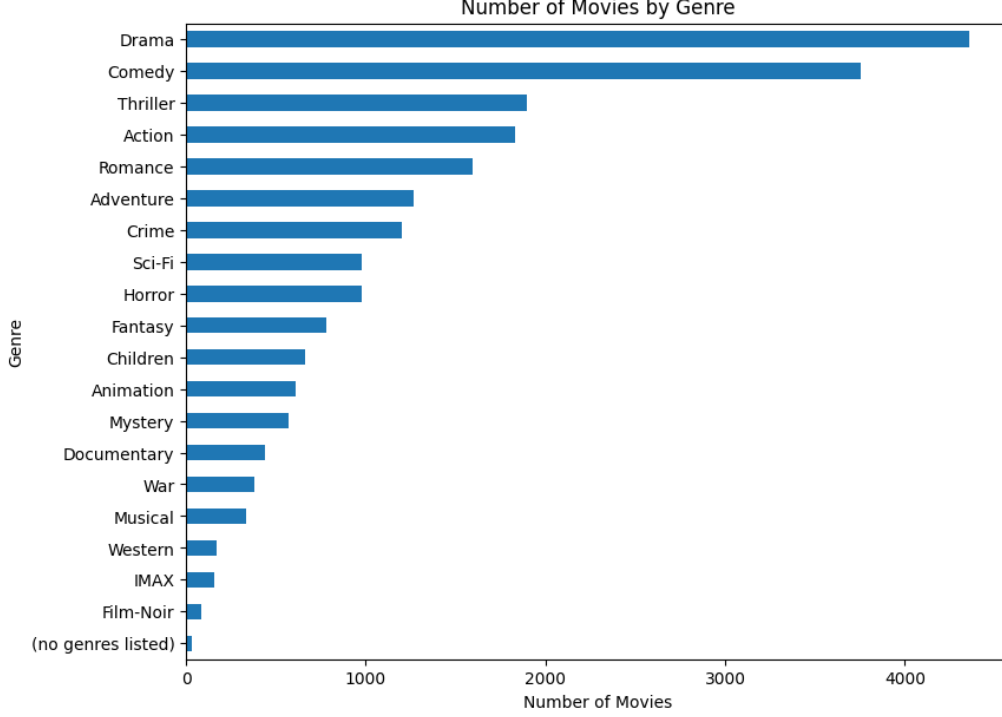


Figure 5: Genre Distribution

5 Experiments

5.1 Experimental Protocol

This section describes the selection of K , the interpretation of the resulting clusters, and a demonstration of the recommendation module. The full pipeline consists of the following steps:

1. **Preprocessing and feature extraction:** Raw ratings and movie genres are transformed into a 610×19 User-Genre matrix of mean-centered genre preference scores. Users with fewer than 10 ratings would be treated as weak-profile users; in this dataset all 610 users satisfy the threshold.
2. **Feature scaling:** The matrix is standardized using `StandardScaler` to zero mean and unit variance per genre column.
3. **Evaluation of $K = 2, \dots, 14$:** K-Means is trained for each K and evaluated using SSE, Silhouette Score, and cluster sizes.
4. **Stability analysis:** Silhouette Score and minimum cluster size are examined across multiple random seeds for the candidate values of K .
5. **Selection of $K = 5$:** $K = 5$ is selected as the best practical trade-off among the criteria above.
6. **Cluster interpretation:** The five clusters are interpreted from their centroids.
7. **Weighted cluster-based recommendation:** Movies are recommended from the user’s cluster using the weighted score of Section 3.6. The recommendation module is evaluated and demonstrated as a downstream use of the clustering result, not as a separate model.

5.2 Selection of K

For each value of K from 2 to 14, K-Means is trained with `random_state = 42` and `n_init = 10`. The number of clusters is evaluated using the Sum of Squared Errors (SSE), Silhouette Score, cluster size, and stability across multiple random seeds.

Sum of Squared Errors (SSE). SSE measures the compactness of the clusters:

$$\text{SSE} = \sum_{k=1}^K \sum_{x_i \in C_k} \|x_i - w_k\|^2, \quad (15)$$

where C_k denotes cluster k and w_k is its centroid. A lower SSE means that users are, on average, closer to their assigned cluster centroids. However, SSE naturally decreases as K increases. Therefore, the Elbow Method is used to identify a reasonable range of K rather than to determine the final value by itself.

Silhouette Score. The Silhouette Score measures both within-cluster cohesion and between-cluster separation. For user i , it is defined as

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}, \quad (16)$$

where $a(i)$ is the mean distance from user i to other users in the same cluster, and $b(i)$ is the smallest mean distance from user i to users in any other cluster. Higher values indicate better cohesion and separation.

Figure 6 shows the SSE and Silhouette Score for $K = 2, \dots, 14$. The SSE decreases gradually as K increases and does not show a single sharp elbow. Therefore, it is considered together with the Silhouette Score and cluster-size analysis rather than being used as the sole selection criterion.

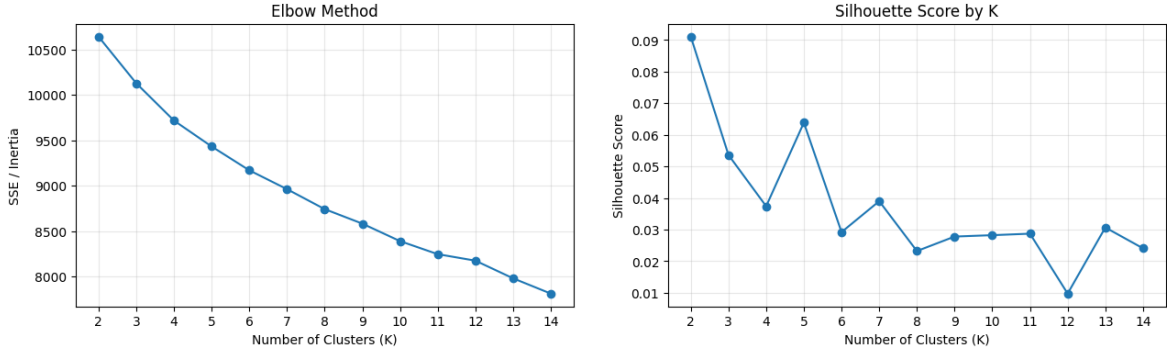


Figure 6: SSE (Elbow Method) and Silhouette Score for $K = 2, \dots, 14$.

In addition to these clustering metrics, cluster sizes are inspected because the clusters are subsequently used by the recommendation module. Extremely small clusters contain too few users to provide sufficient information for cluster-based movie recommendation. The stability of candidate values of K is therefore also evaluated across multiple random seeds.

Table 8 reports the single-run results with `random_state = 42`.

Table 8: K-Means results for different values of K (single run, `random_state` = 42)

K	SSE	Min cluster size	Max cluster size
2	10,639.9	228	382
3	10,126.9	133	270
4	9,716.5	55	255
5	9,430.9	25	279
6	9,168.6	17	229
7	8,961.1	24	200
8	8,741.3	1	193
10	8,389.0	1	172
12	8,173.8	1	168
14	7,811.4	1	173

The single-run results show that increasing K continuously reduces SSE, as expected. However, larger values of K also tend to produce increasingly small clusters. In particular, $K = 8, 10, 12$, and 14 already produce singleton clusters in the single run. This fragmentation is undesirable for the downstream recommendation task because recommendations are generated using ratings from other users in the same cluster.

Based on the initial analysis, moderate values $K = 5, \dots, 8$ are examined further using multiple random seeds. Table 9 summarizes the mean and standard deviation of the Silhouette Score together with the minimum cluster-size statistics across these runs.

Table 9: Multi-seed stability results for candidate values of K

K	Silhouette Mean	Silhouette STD	Avg. Min Cluster Size	Worst Min Cluster Size
5	0.0429	0.0139	32.8	18
6	0.0264	0.0052	29.3	16
7	0.0288	0.0053	20.2	1
8	0.0262	0.0065	12.2	1

Among the candidate values $K = 5, \dots, 8$, $K = 5$ achieves the highest mean Silhouette Score (0.0429) across the tested random seeds. It also maintains usable cluster sizes. In the selected final run, the five clusters contain 107, 279, 63, 25, and 136 users. Across the tested random seeds, the average minimum cluster size is 32.8 users, while the worst minimum cluster size remains 18.

In contrast, $K = 7$ and $K = 8$ can produce singleton clusters, with a worst minimum cluster size of 1. Such extremely small clusters provide limited information from other users and are therefore undesirable for stable and interpretable user segmentation and for the downstream cluster-based recommendation module.

Although $K = 2$ achieves the highest Silhouette Score in the single-run analysis, dividing all 610 users into only two clusters provides a very coarse representation of user preference patterns. The objective of this project is not only to maximize cluster separation, but also to obtain meaningful user segments that can support the recommendation stage. Therefore, additional cluster granularity is considered desirable.

Considering the Elbow analysis, Silhouette Score, cluster-size usability, and stability across random seeds, $K = 5$ is selected as the final number of clusters. This value represents a practical trade-off between clustering quality, stability, interpretability, and usefulness for the downstream recommendation task, rather than an absolute mathematical optimum.

5.3 Cluster Interpretation

Figure 7 shows the genre preference profile of each of the five clusters, generated from the final $K = 5$ model. Each centroid represents the average genre preference profile of a group of users, so the clusters can be interpreted as different movie taste groups.

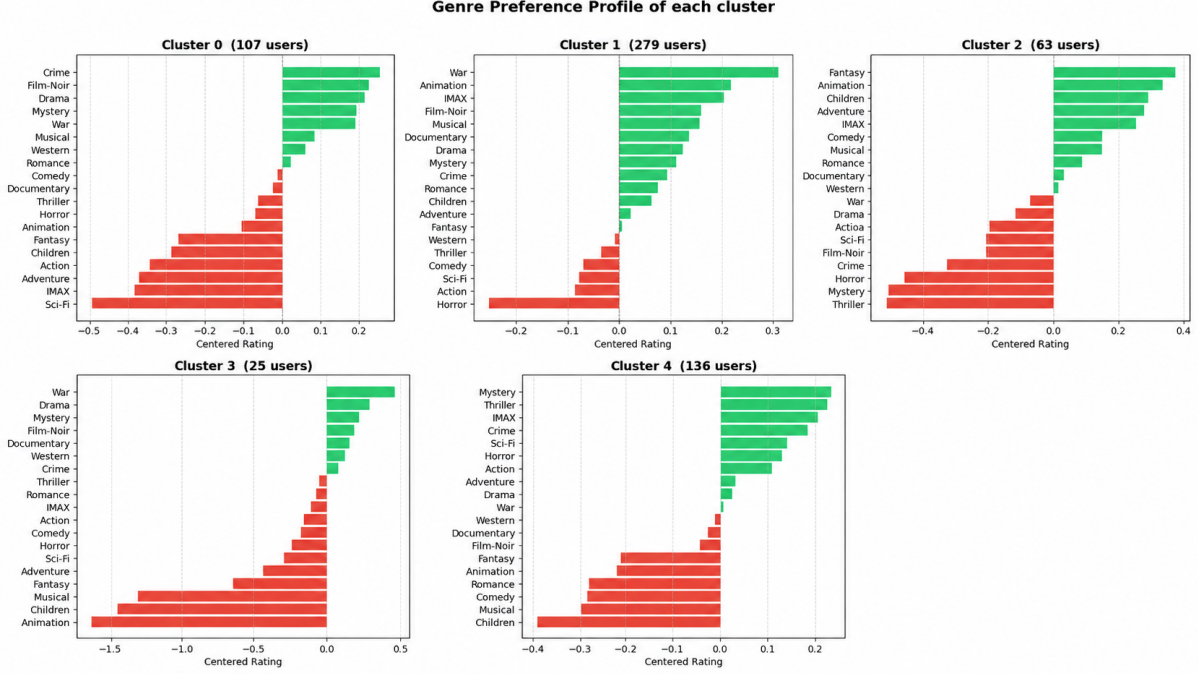


Figure 7: Genre preference profile of each of the five clusters

5.4 Recommendation Demonstration

To demonstrate the downstream application of the clustering result, an existing active user is selected as an example. User 50 has rated 310 movies, has an average rating of 2.78, and is assigned by K-Means to Cluster 0.

Following the recommendation procedure described in Section 3.6, movies already watched by the user are excluded. Candidate movies rated by other users in Cluster 0 are then ranked using the weighted score

$$\text{score}(m) = \text{avg_rating}(m) \log(1 + \text{rating_count}(m)). \quad (17)$$

Table 10 presents the top five recommendations generated for User 50.

Table 10: Top-5 movie recommendations for User 50 (Cluster 0)

Title	Genres	Avg. Rating	Count	Score
The Usual Suspects (1995)	Crime, Mystery, Thriller	4.239	46	16.321
Schindler’s List (1993)	Drama, War	4.070	50	16.003
American Beauty (1999)	Drama, Romance	4.154	39	15.323
One Flew Over the Cuckoo’s Nest (1975)	Drama	4.288	33	15.121
Seven (1995)	Mystery, Thriller	4.066	38	14.895

The example shows how the clustering result is used by the recommendation module. Rather than ranking movies globally, the system first restricts the candidate source to users belonging to the same cluster as the target user. The final ranking then considers both the average rating of each movie and the amount of rating support within that cluster. Thus, the recommendation stage acts as a downstream application of the user segmentation produced by K-Means.

5.5 Clustering Evaluation

The final K-Means model with $K = 5$ is evaluated in terms of clustering quality, cluster-size stability, and interpretability.

- **Silhouette Score:** Across the tested random seeds, $K = 5$ achieves a mean Silhouette Score of 0.0429 with a standard deviation of 0.0139, which is the highest mean value among the candidate solutions $K = 5, \dots, 8$ (Table 9). However, the absolute Silhouette values remain low. This indicates that user preference groups overlap substantially rather than forming sharply separated clusters. Therefore, the clustering result should be interpreted as a useful segmentation of preference patterns rather than as evidence of strongly separated natural groups.
- **Cluster-size stability:** In the selected final run, the five clusters contain 107, 279, 63, 25, and 136 users, respectively, and no singleton cluster is produced. Across the tested random seeds, the average minimum cluster size for $K = 5$ is 32.8 users, while the worst minimum cluster size remains 18 (Table 9). This is important for the downstream recommendation module because each cluster must contain enough users to provide sufficient within-cluster rating information.
- **Interpretability:** Each cluster centroid represents an average centered genre-preference profile. Positive centroid values indicate genres rated above the users’ own average rating level, whereas negative values indicate genres rated below their average. As discussed in Section 5.3, the five centroids exhibit different genre-preference patterns, allowing the clusters to be interpreted as distinct user taste segments.

Overall, the final clustering does not produce strongly separated user groups, as reflected by the relatively low Silhouette Score. Nevertheless, $K = 5$ provides stable and usable cluster sizes together with interpretable genre-preference profiles. These properties make the clustering result suitable for its intended role in this project: user segmentation that supports the downstream cluster-based recommendation module.

5.6 Limitations

- **Timestamp:** User preferences evolve over time, yet the model treats all ratings equally regardless of when they were given. A rating from several years ago carries the same weight as a recent one, which may not accurately reflect a user’s current taste.
- **Genre distribution imbalance:** Some genres (e.g., Drama, Comedy) are significantly more represented than others in the dataset, which may bias the clustering toward dominant genres and reduce the quality of recommendations for users who prefer niche genres.
- **Tags:** The model relies solely on genre information. User-generated tags, which often capture finer-grained preferences (e.g., “atmospheric”, “thought-provoking”), are not currently used.
- **Cold start:** The final system supports new users, but their initial profile is approximate because it is based only on their selected favorite genres. Users who provide no favorite genres, and weak-profile users, receive a popularity-based fallback rather than a personalized recommendation.
- **Cluster separation:** The low Silhouette values indicate that taste groups overlap, so cluster assignments should be understood as approximate segments.

5.7 Future Work

- **Timestamp:** Incorporate time-aware weighting so that recent ratings contribute more to the user profile than older ones, better capturing the dynamic nature of user preferences.
- **Genre distribution imbalance:** Apply resampling or weighting strategies to reduce the dominance of overrepresented genres, ensuring that users with niche tastes are clustered and recommended more accurately.
- **Tags:** Integrate user-generated tags as additional features alongside genre information, enabling the model to capture richer and more nuanced preference signals.
- **Cold start:** Develop richer onboarding signals than a list of favorite genres to build better initial profiles for new users.
- **Recommendation evaluation:** Design a stronger quantitative evaluation of the recommendation module.

References

- [1] Harper, F. M., & Konstan, J. A. (2015). The MovieLens datasets: History and context. *ACM Transactions on Interactive Intelligent Systems*, 5(4), Article 19. <https://doi.org/10.1145/2827872>
- [2] Lee, J. H., Kim, D. H., Lee, Y. H., & Hwang, I. (2012). A literature review and classification of recommender systems research. *Expert Systems with Applications*, 39(11), 10059–10072. <https://doi.org/10.1016/j.eswa.2012.02.038>